Aplicación Multi-Base de Datos para una Red de Instituciones Médicas

Entrega II — Documento comparativo, modelo corregido y modelo documental

Raúl Esdras Aarón Gutiérrez Ajucum - 1503223
Ingeniería en Informática y Sistemas, Universidad Rafael Landívar
Bases de Datos II

I. ASIGNACIÓN DE MOTORES Y JUSTIFICACIÓN

La aplicación implementa las mismas reglas de negocio sobre cuatro motores distintos, con un motor por institución y sin repetir ninguno. La asignación no es arbitraria: responde al volumen de datos, a la estructura de la información, a la necesidad de transacciones estrictas y al personal técnico disponible. La Tabla 1 resume la decisión y la justificación.

Tabla 1
Asignación de motores por institución

Institución	Motor	Adaptador	Justificación
Hospital General	PostgreSQL	`adapters/postgres`	Integridad transaccional fuerte: un diagnóstico no puede quedar a medias. PostgreSQL ofrece ACID, claves foráneas, restricciones UNIQUE y bloqueo de filas con `FOR UPDATE`. El modelo es altamente relacional: cita, diagnóstico, tratamiento, internamiento, factura y seguro.
Centro Médico Pediátrico	MySQL (InnoDB)	`adapters/mysql`	Consultas frecuentes y cortas, con esquema estable: paciente, consulta, vacunación y crecimiento. InnoDB mantiene transacciones y claves foráneas, con buen rendimiento en lecturas y escrituras simples.
Laboratorio Clínico	MongoDB	`adapters/mongo`	Los resultados cambian de estructura según el examen (hematología, orina, cultivo). Un documento flexible evita columnas nulas o un modelo entidad-atributo-valor.
Clínica Odontológica	Firebase (Firestore)	`adapters/firebase`	No cuenta con personal técnico propio: Firestore es administrado, no requiere servidores ni respaldos propios y escala automáticamente. El volumen es pequeño y las secretarias priorizan la rapidez de atención.

El enrutamiento se resuelve una sola vez al iniciar sesión. El token JWT contiene la institución y el rol; `repositoryFactory.js` traduce la institución al adaptador correspondiente y `HistorialService` trabaja únicamente contra el puerto. Por eso el frontend y las reglas de negocio no cambian cuando cambia el motor.

La asignación cumple las restricciones del proyecto: se usan cuatro motores diferentes y cada uno se utiliza una sola vez. La justificación se amplía en las secciones siguientes.

II. MODELADO DE BASE DE DATOS CORREGIDO

El modelo relacional de la Entrega I se revisó contra la implementación final. El modelo corregido está en `database/postgres.sql` y `database/mysql.sql`, ambos en tercera forma normal (3FN). Esta sección describe las entidades, las relaciones y los cambios principales.

II.A Hospital General — PostgreSQL

El Hospital General es el modelo más completo: cubre citas, internamientos, diagnósticos, tratamientos, administración de medicamentos, signos vitales, camas, facturación y seguros. La Tabla 2 enumera las entidades y sus relaciones principales.

Tabla 2
Entidades del Hospital General (PostgreSQL)

Entidad	Clave primaria	Relaciones y propósito
paciente	id	1:N con cita, diagnostico, internamiento, factura y seguro_medico; N:M con alergia por paciente_alergia. paciente.seguro_medico_id apunta a la póliza vigente. Incluye baja lógica (activo, motivo_baja, detalle_baja, fecha_baja).
empleado	id	1:N con cita (médico), diagnostico, internamiento, pago_medico, administracion_medicamento y signos_vitales. Distingue médicos, enfermeros y administrativos.
especialidad	id	Catálogo; 1:N con empleado y cita.
cita	id	N:1 con paciente, empleado y especialidad; 1:N con diagnostico; N:1 con pago_medico. Registra fecha, motivo, estado y observaciones.
diagnostico	id	N:1 con paciente, empleado, cita e internamiento; 1:N con tratamiento y receta_medicamento.
tratamiento	id	N:1 con diagnostico. Descripción, fecha de inicio, fecha de fin y estado (en curso, finalizado, suspendido).
medicamento	id	Catálogo; 1:N con receta_medicamento y administracion_medicamento.
receta_medicamento	id	N:1 con diagnostico y medicamento. Dosis, frecuencia, duración y fecha.
internamiento	id	N:1 con paciente, cama_habitacion y empleado; 1:N con signos_vitales, administracion_medicamento y factura.
cama_habitacion	id	1:N con internamiento. Habitación, cama, tipo y estado; restricción única (numero_habitacion, numero_cama).
signos_vitales	id	N:1 con internamiento y empleado. Temperatura, presión, frecuencias y saturación.
administracion_medicamento	id	N:1 con internamiento, medicamento y empleado. Dosis, vía, fecha y observaciones.
alergia	id	Catálogo; N:M con paciente por paciente_alergia (severidad y fecha de registro).
paciente_alergia	(paciente_id, alergia_id)	Resuelve la N:M; severidad y fecha de registro.
seguro_medico	id	N:1 con paciente; aseguradora, número de póliza y cobertura. paciente.seguro_medico_id apunta al seguro vigente.
factura	id	N:1 con paciente, internamiento y cita; concepto, monto, fecha y estado_pago (pendiente, pagado, anulado). El nombre del paciente se obtiene con JOIN; no se duplica.
pago_medico	id	N:1 con empleado; periodo, tipo de pago, cantidad de consultas, monto, fecha y estado. 1:N con cita.

El modelo está en 3FN: cada tabla tiene clave primaria, los atributos no clave dependen de la clave completa y no hay dependencias transitivas. Los catálogos (especialidad, medicamento, alergia) evitan repetir texto; paciente_alergia y receta_medicamento resuelven relaciones N:M; factura guarda solo identificadores y montos. Se agregaron índices para el historial: cita(paciente_id, fecha_hora DESC), diagnostico(paciente_id), tratamiento(diagnostico_id), factura(paciente_id, fecha DESC), seguro_medico(paciente_id) y cama_habitacion(estado, tipo). paciente.dpi es UNIQUE.

II.B Centro Médico Pediátrico — MySQL

El Centro Pediátrico prioriza consultas frecuentes y cortas, control de crecimiento, vacunación y controles programados. La Tabla 3 enumera sus entidades.

Tabla 3
Entidades del Centro Médico Pediátrico (MySQL)

Entidad	Clave primaria	Relaciones y propósito
paciente	id	Datos demográficos, responsable (nombre_padre_madre), contacto y baja lógica. 1:N con consulta, esquema_vacunacion, curva_crecimiento, control_programado y paciente_alergia.
pediatra	id	1:N con consulta y pago_pediatra.
consulta	id	N:1 con paciente y pediatra; 1:N con diagnostico; N:1 con pago_pediatra. Fecha, motivo, peso, talla y observaciones.
diagnostico	id	N:1 con consulta; 1:N con tratamiento y receta_medicamento.
tratamiento	id	N:1 con diagnostico; descripción, fechas y estado.
medicamento	id	Catálogo; 1:N con receta_medicamento.
receta_medicamento	id	N:1 con diagnostico y medicamento; dosis, frecuencia, duración y fecha.
alergia	id	Catálogo; N:M con paciente por paciente_alergia.
paciente_alergia	(paciente_id, alergia_id)	Severidad.
vacuna	id	Catálogo; 1:N con esquema_vacunacion.
esquema_vacunacion	id	N:1 con paciente y vacuna; número de dosis, fecha, lote y próxima dosis.
curva_crecimiento	id	N:1 con paciente; fecha, peso, talla, perímetro cefálico y percentiles.
control_programado	id	N:1 con paciente; fecha, tipo y estado.
pago_pediatra	id	N:1 con pediatra; periodo, tipo, cantidad, monto, fecha y estado; 1:N con consulta.

También está en 3FN. paciente.nombre_padre_madre representa un único responsable; si el centro necesitara varios tutores, se agregaría una tabla tutor y una relación N:M. Se agregaron índices consulta(paciente_id, fecha) y tratamiento(diagnostico_id). Las tablas diagnostico, medicamento, receta_medicamento y tratamiento se incorporaron en la Entrega II porque el modelo de la Entrega I no incluía diagnóstico ni medicamento y las reglas de negocio los exigen.

II.C Cambios respecto a la Entrega I

La Tabla 4 resume las correcciones y extensiones del modelo. La corrección no consistió solo en agregar tablas: también se revisó la normalización y se eliminó la duplicación del nombre del paciente en facturas y pagos.

Tabla 4
Cambios y extensiones del modelo relacional

Cambio	Motor	Motivo
Baja lógica del paciente (activo, motivo_baja, detalle_baja, fecha_baja)	PostgreSQL y MySQL	Archivar el historial por traslado, fallecimiento, causa administrativa u otra; el historial archivado queda en solo lectura.
Observaciones de la cita (cita.observaciones)	PostgreSQL	Registrar notas clínicas de la atención.
Diagnóstico, medicamento, receta y tratamiento	MySQL	La Entrega I no tenía diagnóstico ni medicamento; se requieren para registrar consultas completas.
Tratamiento explícito	PostgreSQL	Mantener tratamientos con descripción, fechas y estado, ligados al diagnóstico.
Receta de medicamentos	PostgreSQL	Ligar cada receta al diagnóstico y al catálogo de medicamentos.
Camas disponibles	PostgreSQL	Consultar camas libres por tipo y habitación; se siembran camas de demostración si la tabla está vacía.
Facturación y seguros	PostgreSQL	Registrar facturas, cambiar su estado y administrar la póliza del paciente; el nombre se muestra con JOIN.
Índices de historial	PostgreSQL y MySQL	Acelerar consultas por paciente, tratamientos, facturas, seguros y camas.

El nombre del paciente no se almacena en factura ni en pago_medico: se obtiene con JOIN. Esto evita redundancia y anomalías de actualización, y mantiene la 3FN. La clasificación de la información se conserva: datos públicos (catálogos, disponibilidad de camas), confidenciales

(contacto, facturación, pólizas, honorarios) y restringidos (diagnósticos, tratamientos, alergias, resultados, signos vitales).

III. MODELO DOCUMENTAL PARA MONGODB Y FIREBASE

Las instituciones no relacionales no usan tablas: usan colecciones y documentos. La decisión de embeber o referenciar se apoya en cinco criterios: crecimiento esperado, si el dato se lee siempre con el padre, necesidad de atomicidad, límite de tamaño del documento (16 MB en MongoDB y 1 MB en Firestore) y frecuencia de actualización.

III.A Laboratorio Clínico — MongoDB

El laboratorio procesa resultados cuya estructura cambia según el examen. El modelo usa cuatro colecciones: `pacientes`, `profesionales`, `consultas` y `resultados`.

- `pacientes`: un documento por paciente, con datos estables y `_id` como referencia.
- `profesionales`: médicos remitentes y bioanalistas.
- `consultas`: una orden o atención por documento. Embebe `diagnosticos`; cada diagnóstico embebe `medicamentos` y `tratamientos`. Son pocos por consulta, nacen y se leen juntos, y cada alta es una escritura atómica sobre un solo documento.
- `resultados`: colección separada, referenciada por `pacienteId`. Cada documento guarda `examen`, `fecha` y `campos`, un objeto con la estructura variable del examen. Un paciente acumula cientos de resultados; embeberlos haría crecer el documento sin límite (tope de 16 MB) y obligaría a leerlos siempre.

Se crearon índices `pacientes {apellido, nombre}`, `consultas {pacienteId, fecha:-1}` y `resultados {pacienteId, fecha:-1}`. Las colecciones `consultas` y `resultados` tienen validador `$jsonSchema` para exigir los campos obligatorios y el tipo de dato.

```
{
  _id, pacienteId, profesionalId, fecha, motivo,
  diagnosticos: [{
    _id, codigo, descripcion, tipo, fecha,
    medicamentos: [{ nombre, dosis, frecuencia, duracion }],
    tratamientos: [{ descripcion, fechaInicio, fechaFin, estado }]
  }]
}

{ _id, pacienteId, examen, fecha, campos: { ... } }
```

III.B Clínica Odontológica — Firebase (Firestore)

Firestore organiza los datos en colecciones y subcolecciones. La clínica no tiene personal técnico, así que el servicio administrado reduce la operación.

- `pacientes/{id}`: documento del paciente.
- `pacientes/{id}/consultas/{id}`: cita; embebe `diagnosticos`, y cada diagnóstico embebe `medicamentos` y `tratamientos`.
- `pacientes/{id}/procedimientos/{id}`: procedimientos dentales en subcolección; se referencian porque crecen con los años y se paginan aparte.
- `profesionales/{id}`: odontólogos.

La escritura de diagnósticos, medicamentos y tratamientos usa `runTransaction`: lee la consulta, modifica el arreglo `diagnosticos` y escribe todo de nuevo de forma atómica, con reintento automático ante conflictos. Los procedimientos se crean con una escritura independiente. La búsqueda por nombre se filtra en memoria porque Firestore no hace búsqueda parcial de texto; el volumen de una clínica pequeña lo permite.

```
pacientes/{pacienteId}
  nombre, apellido, fechaNacimiento, genero, telefono, direccion, activo
  consultas/{consultaId}
    profesionalId, fecha, motivo, observaciones,
    diagnosticos: [{ id, codigo, descripcion, tipo, fecha,
                     medicamentos: [...], tratamientos: [...] }]
  procedimientos/{procedimientoId}
    fecha, datos: { piezaDental, descripcion, costo }
```

III.C Matriz de decisión

La Tabla 5 resume, para cada dato, si se embebe o se referencia y por qué.

Tabla 5
Decisión de embeber o referenciar en MongoDB y Firebase

Dato	MongoDB	Firebase	Decisión	Motivo
Datos del paciente	`pacientes`	`pacientes/{id}`	Referencia	Estable, se consulta por identificador y es único por paciente.
Consultas	`consultas`	Subcolección `consultas`	Referencia	Crecen con los años; se leen por paciente y se paginan.
Diagnósticos	Embebidos en la consulta	Embebidos en la consulta	Embeber	Pocos por consulta; se leen junto con ella y la escritura es atómica.
Medicamentos	Embebidos en el diagnóstico	Embebidos en el diagnóstico	Embeber	Pocos; se leen con el diagnóstico y se agregan en la misma escritura.
Tratamientos	Embebidos en el diagnóstico	Embebidos en el diagnóstico	Embeber	Pocos; se leen con el diagnóstico y se agregan en la misma escritura.
Resultados de examen (Laboratorio)	`resultados`	No aplica	Referencia en MongoDB	Cientos por paciente, estructura variable y límite de 16 MB.
Procedimientos dentales (Clínica)	No aplica	Subcolección `procedimientos`	Referencia en Firestore	Crecen con los años, se paginan aparte y evitan el límite de 1 MB.
Profesionales	`profesionales`	`profesionales`	Referencia	Catálogo compartido e independiente del paciente.

IV. CONSISTENCIA

Las reglas de negocio se aplican en `HistorialService`, antes de llamar al adaptador: un paciente archivado es de solo lectura; no se receta un medicamento al que el paciente es alérgico (en las instituciones que registran alergias); la fecha de fin de un tratamiento no puede ser anterior a la de inicio; la fecha de nacimiento no puede ser futura; los campos obligatorios dependen de la institución; las facturas y los seguros solo los gestiona el rol administrativo del Hospital General. La diferencia entre motores está en cómo se garantiza la atomicidad.

IV.A PostgreSQL

Las escrituras compuestas usan `_tx(fn)`: se toma un cliente dedicado, se ejecuta `BEGIN`, se corre la operación y se hace `COMMIT`; si algo falla, se ejecuta `ROLLBACK` y se libera el cliente. Se usan transacciones para registrar una cita, agregar un diagnóstico, recetar un medicamento, agregar un tratamiento, registrar una alergia, registrar un pago, vincular citas a un pago y guardar un seguro. Antes de modificar un diagnóstico o una cita se usa `SELECT … FOR UPDATE` para bloquear la fila padre y evitar que dos usuarios agreguen datos al mismo tiempo. Las claves foráneas con `RESTRICT`, `CASCADE` y `SET NULL` mantienen la integridad referencial; `UNIQUE` protege el DPI y la combinación habitación/cama. Los estados se validan en el servicio.

IV.B MySQL

MySQL usa el mismo patrón con `beginTransaction`, `commit` y `rollback` sobre una conexión del pool. Las operaciones compuestas (consulta, diagnóstico, receta, tratamiento, extras, pagos y vinculación) son transaccionales. `FOR UPDATE` bloquea la consulta o el diagnóstico padre; `INSERT IGNORE` y `ON DUPLICATE KEY UPDATE` mantienen los catálogos y las alergias sin duplicados. InnoDB aporta claves foráneas y transacciones ACID.

IV.C MongoDB

En MongoDB la unidad de atomicidad es el documento. Agregar un diagnóstico es un `updateOne` con `$push` sobre `consultas`; agregar un medicamento o un tratamiento es un `updateOne` con `$push` sobre `diagnosticos.$.medicamentos` o `diagnosticos.$.tratamientos`. Como todo lo relacionado vive en el documento de la consulta, no se necesita una transacción multi-documento. Los resultados son documentos independientes en `resultados`; su creación es una escritura atómica y no comparte invariantes con la consulta. Los validadores `$jsonSchema` y los índices complementan la consistencia.

IV.D Firebase

Firestore usa `runTransaction` para leer la consulta, modificar el arreglo `diagnosticos` y escribirla de nuevo de forma atómica; si hay conflicto, el SDK reintenta. Así se agregan diagnósticos, medicamentos y tratamientos sin perder actualizaciones concurrentes. La creación de pacientes y procedimientos son escrituras de un solo documento, también atómicas. Las subcolecciones mantienen los documentos por debajo del límite de 1 MB. No se requieren transacciones entre documentos para las reglas actuales.

IV.E Resumen

Tabla 6
Mecanismos de consistencia por motor

Motor	Unidad de atomicidad	Mecanismo	Operaciones
PostgreSQL	Transacción multi-fila	`BEGIN/COMMIT/ROLLBACK`, `FOR UPDATE`, claves foráneas	Cita, diagnóstico, receta, tratamiento, alergia, pago, seguro.
MySQL	Transacción multi-fila	`beginTransaction/commit/rollback`, `FOR UPDATE`, claves foráneas	Consulta, diagnóstico, receta, tratamiento, extras, pago.
MongoDB	Documento	`$push` / `updateOne` atómico	Diagnóstico, medicamento y tratamiento dentro de la consulta.
Firebase	Documento / transacción	`runTransaction`	Diagnóstico, medicamento y tratamiento dentro de la consulta.

El servicio aplica las mismas validaciones en los cuatro motores; los adaptadores solo cambian la forma de persistir. Esa separación es la que permite afirmar que las reglas de negocio son equivalentes.

V. SEGURIDAD

La seguridad se aplica en capas: autenticación, autorización, aislamiento por institución, clasificación de la información y protección específica de cada motor.

V.A Autenticación y sesión

- Usuario y contraseña; la contraseña se guarda con `bcrypt` (10 rondas) y nunca en texto plano.
- Sesión con JWT firmado, expiración de 4 horas. El token incluye `sub`, `nombre`, `institucion` y `rol`.
- Límite de 10 intentos de login por minuto y por IP; al superarlo se responde 429.
- Credenciales inválidas devuelven 401 genérico, sin revelar si el usuario existe.

V.B Autorización por rol

- `medico`: registra consultas, diagnósticos, medicamentos, tratamientos y extras; ve el historial clínico completo.
- `administrativo`: en las instituciones que lo soportan, gestiona pagos, facturas y seguros; no ve información clínica restringida. En el historial se eliminan observaciones, diagnósticos y extras.

• Solo el administrativo del Hospital General gestiona facturas y seguros; solo los administrativos de Hospital General y Centro Pediátrico gestionan pagos de honorarios.

• `_soloMedico()` y `_soloAdministrativo()` aplican la restricción en el servicio; los adaptadores rechazan módulos no soportados con 400.

V.C Aislamiento por institución y enrutamiento

• El token define la institución; `repositoryFactory.js` resuelve el adaptador. El usuario no elige el motor.

• Cada institución tiene su propia base y sus propias credenciales; no hay consultas cruzadas.

• El frontend recibe la configuración de la institución en el login y en `/api/me`; los campos y etiquetas se generan desde esa configuración, no desde el motor.

V.D Clasificación de la información

Tabla 7
Clasificación de la información y control de acceso

Clasificación	Ejemplos	Control
Pública	Catálogos (especialidades, vacunas, medicamentos), disponibilidad de camas.	Usuarios autenticados; no contiene datos personales.
Confidencial	Facturación, pólizas, honorarios y datos de contacto.	Facturación, pólizas y honorarios: rol administrativo. El rol clínico ve el contacto necesario para la atención.
Restringida	Diagnósticos, tratamientos, alergias, resultados, signos vitales, administración de medicamentos.	Solo rol clínico; el administrativo recibe el historial redactado.

La baja del paciente también es información administrativa: se registra el motivo (`traslado`, `fallecimiento`, `administrativa`, `otra`) y el detalle, y el historial queda en solo lectura.

V.E Protección por motor

• PostgreSQL: consultas parametrizadas (`$1`), `FOR UPDATE`, claves foráneas y `UNIQUE`; en despliegue, usuario de base con privilegios mínimos y TLS; secretos en variables de entorno.

• MySQL: consultas parametrizadas (`?`), transacciones InnoDB, claves foráneas; en despliegue, usuario con privilegios mínimos y TLS.

• MongoDB: `ObjectId` tipados, expresión regular escapada en la búsqueda, validadores `$jsonSchema`; acceso solo desde el servidor; en despliegue, usuario con privilegios mínimos y TLS.

• Firebase: acceso solo con el Admin SDK desde el servidor; en despliegue, reglas de Firestore `allow read, write: if false` para clientes; cuenta de servicio en un archivo ignorado por Git; infraestructura administrada.

• Frontend: el texto se escapa antes de pintarse (anti-XSS); el cuerpo JSON se limita a 100 KB; los errores devuelven mensajes genéricos y no exponen trazas; si la base no responde se devuelve 503.

• Secretos: `.env` y `firebase-service-account.json` están en `.gitignore` y no se incluyen en el paquete de entrega.

VI. OPERACIONES EQUIVALENTES EN LOS CUATRO MOTORES

La Tabla 8 relaciona las operaciones principales con el endpoint y la implementación en cada motor. El frontend es el mismo; la configuración de la institución cambia las etiquetas y los módulos disponibles.

Tabla 8

Operaciones equivalentes y módulos por institución

Operación	Endpoint	PostgreSQL / MySQL	MongoDB	Firebase
Iniciar sesión y enrutar	`POST /api/login`	JWT con institución y rol; `repositoryFactory`	Igual	Igual
Listar o buscar pacientes	`GET /api/pacientes`	`ILIKE` / `LIKE`	Regex escapada	Filtro en memoria
Crear paciente	`POST /api/pacientes`	`INSERT`	`insertOne`	`set`
Ver historial	`GET /api/pacientes/:id/historial`	JOINs	Consulta + arreglos embebidos	Subcolecciones
Registrar consulta	`POST /api/pacientes/:id/consultas`	Transacción + `INSERT`	`insertOne`	`set` en subcolección
Agregar diagnóstico	`POST .../diagnosticos`	Transacción + `FOR UPDATE` + `INSERT`	`$push`	`runTransaction`
Recetar medicamento	`POST .../recetas`	Transacción + validación de alergia + `INSERT`	`$push`	`runTransaction`
Agregar tratamiento	`POST .../tratamientos`	Transacción + `INSERT`	`$push`	`runTransaction`
Registrar extra	`POST .../extras/:tipo`	Alergia; vacuna/crecimiento en MySQL	Resultado de examen	Procedimiento dental
Dar de baja o reactivar	`POST .../baja`, `.../reactivar`	`UPDATE`	`updateOne`	`update`
Camas disponibles	`GET /api/camas`	`SELECT` (solo PostgreSQL)	No aplica (400)	No aplica (400)
Facturas y seguros	`GET/POST /api/facturas`, `.../seguro`	`SELECT/INSERT/UPDATE` + JOIN (solo PostgreSQL, admin)	No aplica (400)	No aplica (400)
Pagos de honorarios	`GET/POST /api/pagos`	Transacción + JOIN (PostgreSQL y MySQL, admin)	No aplica (400)	No aplica (400)

Las operaciones 1 a 10 son equivalentes en los cuatro motores. Las operaciones 11 a 13 son módulos propios de una institución y los adaptadores que no los soportan responden 400; el servicio también los bloquea si la institución no los declara. Esto evita que la interfaz muestre módulos que no existen en el motor activo.

VII. ARCHIVOS DE IMPLEMENTACIÓN

El documento se complementa con los siguientes archivos del proyecto:

- `database/postgres.sql`: modelo relacional del Hospital General, con tablas, claves, restricciones e índices.
- `database/mysql.sql`: modelo relacional del Centro Pediátrico, con las tablas agregadas en la Entrega II.
- `database/mongo_init.js`: colecciones, validadores e índices del Laboratorio Clínico.
- `backend/src/domain/ports.js`: contrato único que implementan los cuatro adaptadores.
- `backend/src/application/HistorialService.js`: reglas de negocio comunes.
- `backend/src/infrastructure/repositoryFactory.js`: enrutamiento institución → motor.
- `backend/src/adapters/`: implementaciones PostgreSQL, MySQL, MongoDB y Firebase.
- `frontend/`: interfaz única, generada a partir de la configuración de la institución.
- `README.md`: puesta en marcha y usuarios de demostración.

Con estos archivos se puede reconstruir el modelo, ejecutar la aplicación y verificar que las reglas de negocio son las mismas en los cuatro motores.